Deadlock Problem

17 Oct 2025
CS303 Autumn 2025

Deadlock Problem

- Deadlock: A set of processes is said to be in deadlock if every process is waiting on an event on another process in that set itself
- To solve deadlock, we need to identify the necessary criteria for its occurrence
  - and avoid all such scenarios
- The necessary criteria for Deadlock to occur are:
  - Mutual Exclusion
  - Hold and Wait
  - No preemption of **resources**
  - Circular waiting

Deadlock Problem: Illustrate

- Mutual Exclusion: A printer would print a job on mutually exclusive basis
- Hold and Wait: If there are two resources say a scanner and camera
  - Say scanner if busy a process holds on to camera and waits for scanner
- No preemption of resource:
  - Say Printer cannot be preempted from its current allocation i.e print task
- Cyclic waiting:
  - Typically hold and wait cases if more than two in number result in cyclic waiting

“Deadlock Illustration” via Resource Allocation Graph

- Deadlock problem can be elucidated via graph
- Particularly “Resource Allocation Graph”, is a “Directed Graph”
- RAG <V,E>
  - V: Vertices
    - Two types:
      - Processes depicted as Circles o
      - Resource Types depicted as Boxes []
        - This box would include several dots in it indicating the number of instances of that resource type available
  - E: Edges
    - Two types
      - Request Edge: Directed edge (P_i R_k) from a Process P_i to Resource box R_k
      - Allocation Edge: Directed edge (R_k P_i) from *Resource instance dot* to the Process requesting that resource type, if a resource instance is available
- RAG can be exploited to analyse whether a Deadlock is possible or not

Illustration with RAG

- Consider three processes, so three circular vertices:

Illustration with RAG

- And three resources:

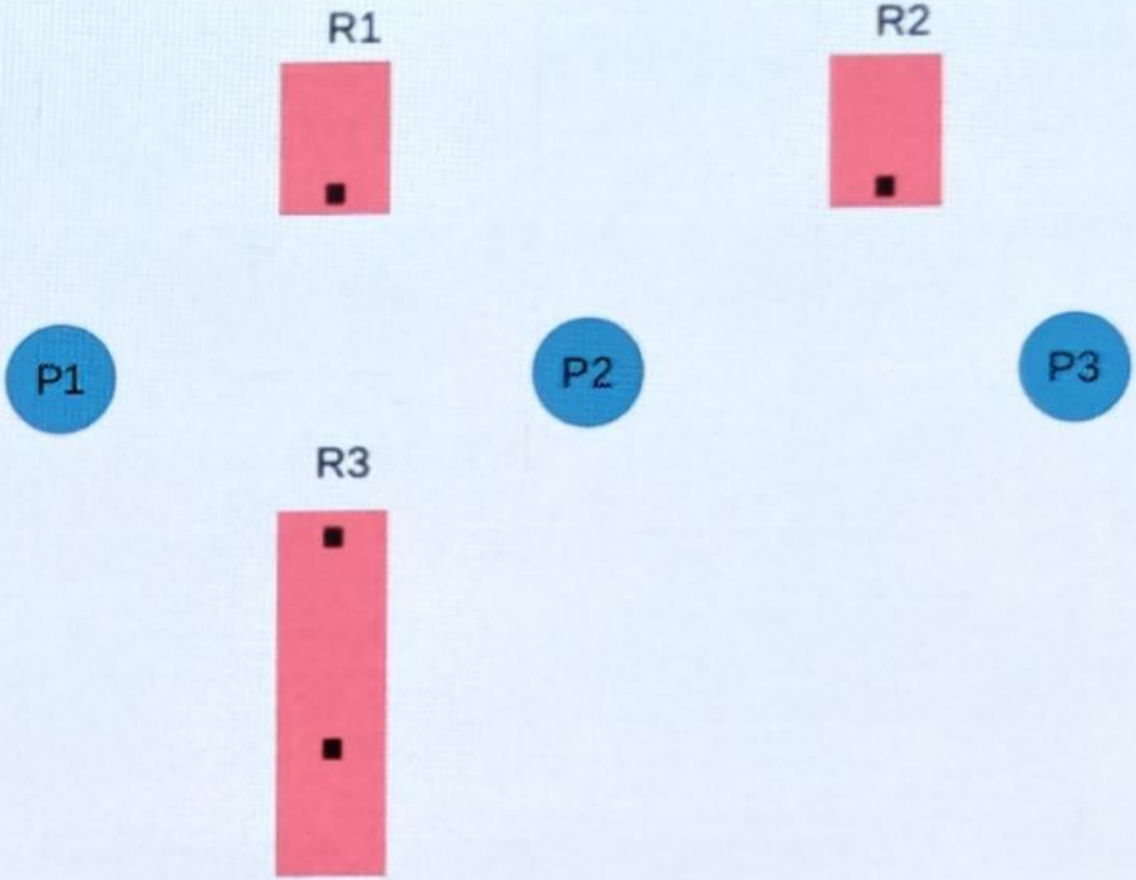


Illustration with RAG /2

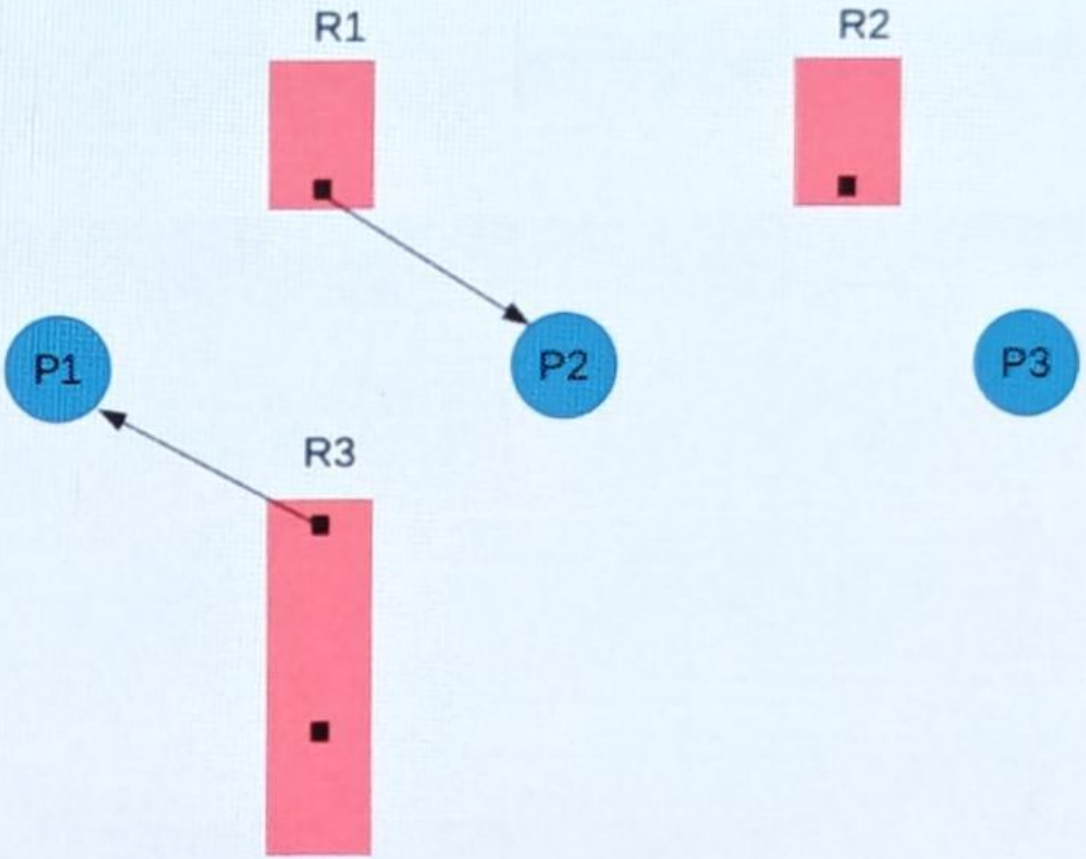
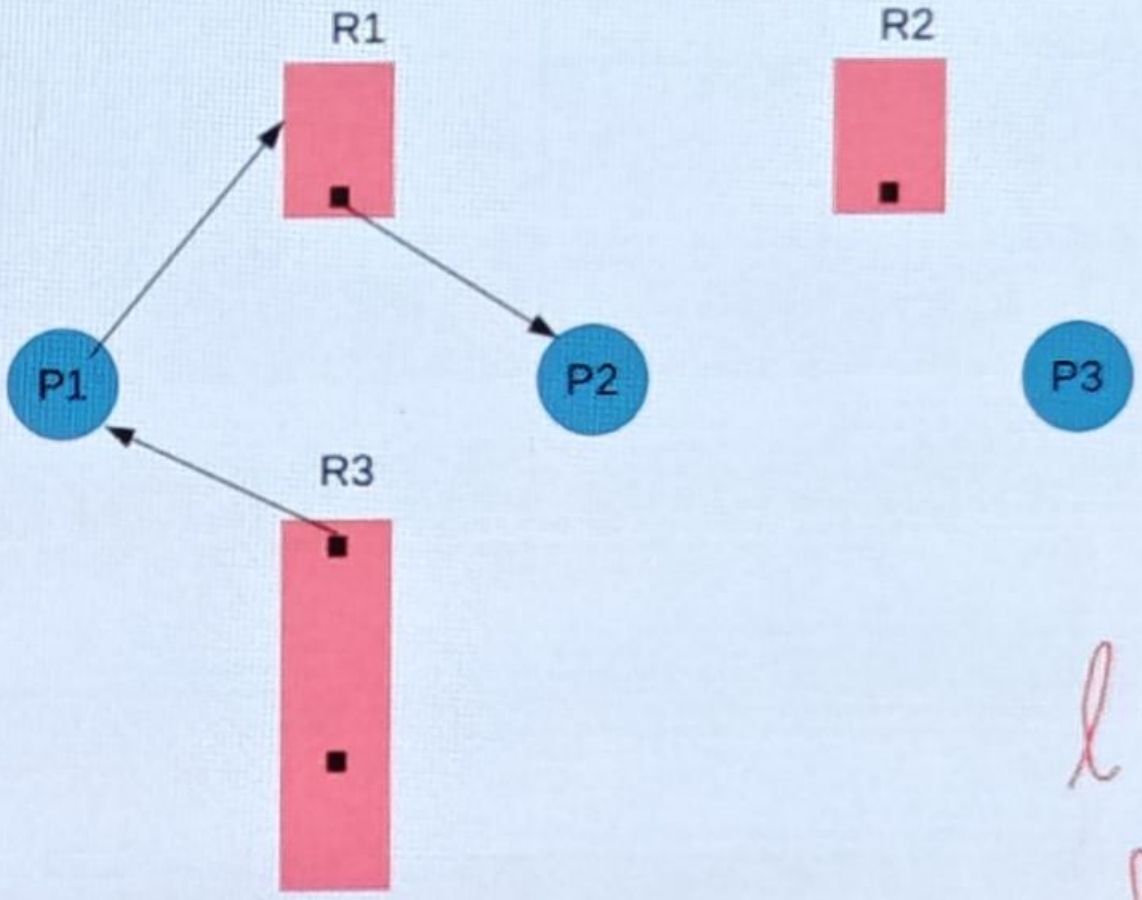


Illustration with RAG /2



$l(R_1)$

$\neg l(R_1)$

Illustration with RAG /2

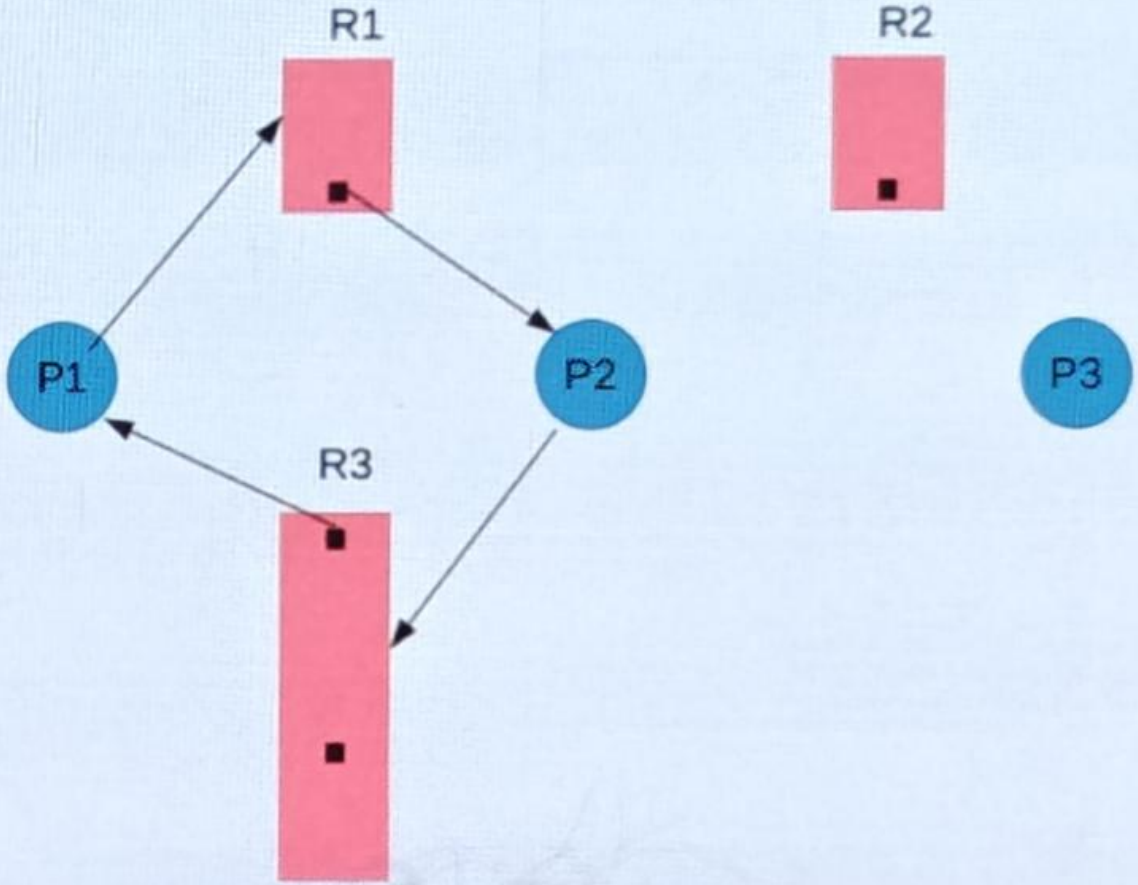


Illustration with RAG /2

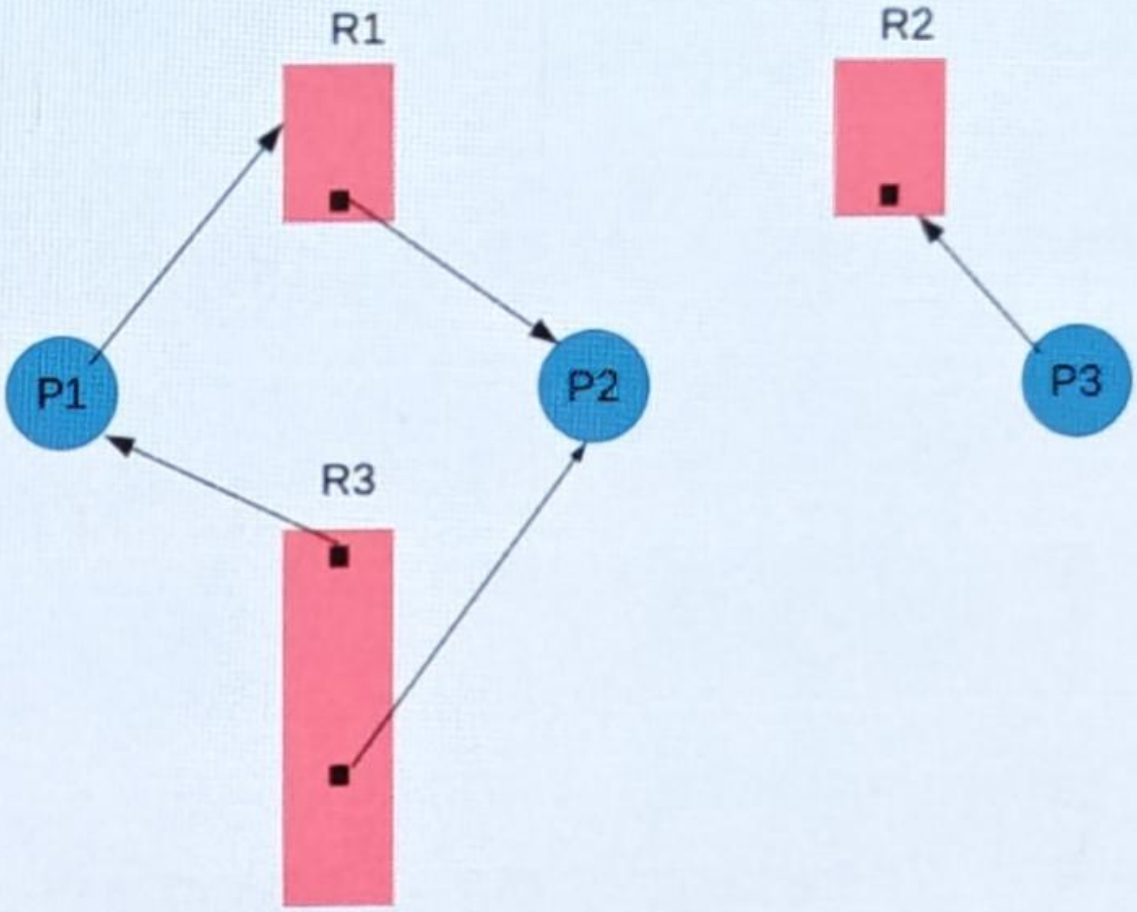


Illustration with RAG /2

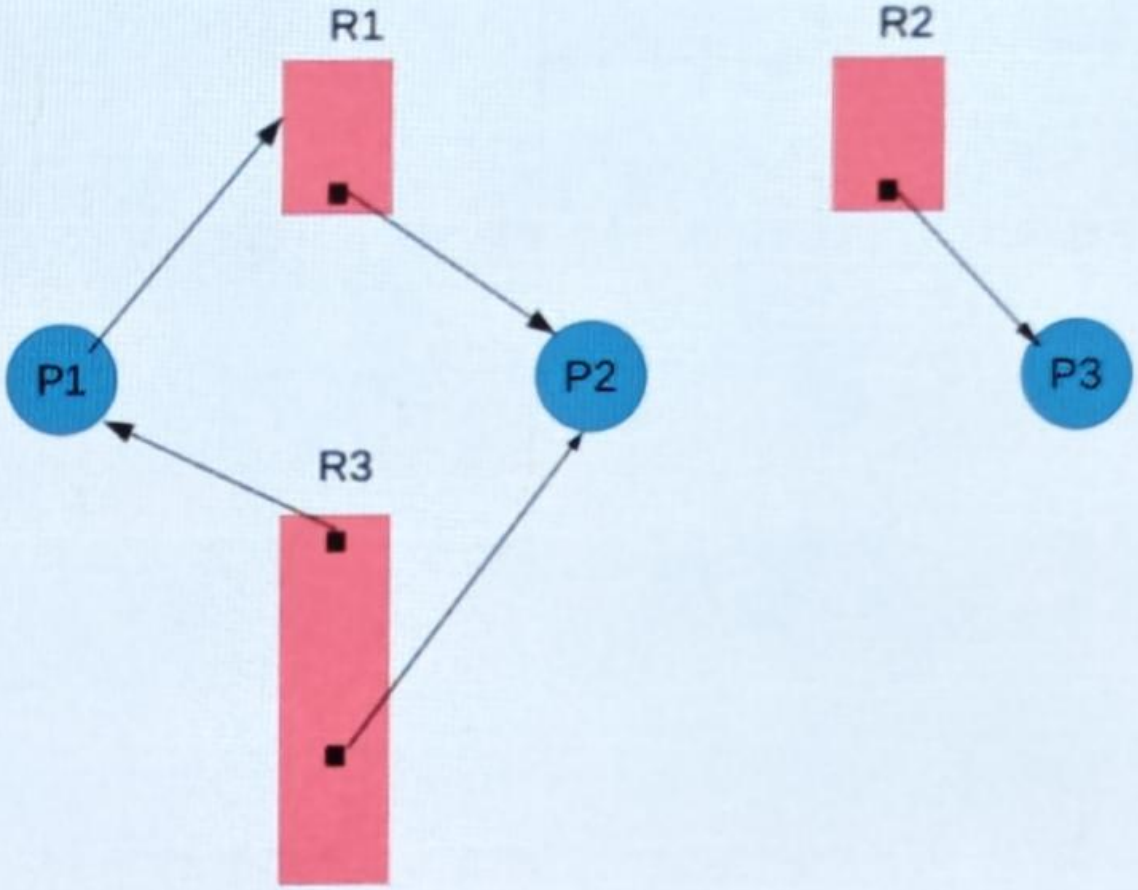


Illustration with RAG /2

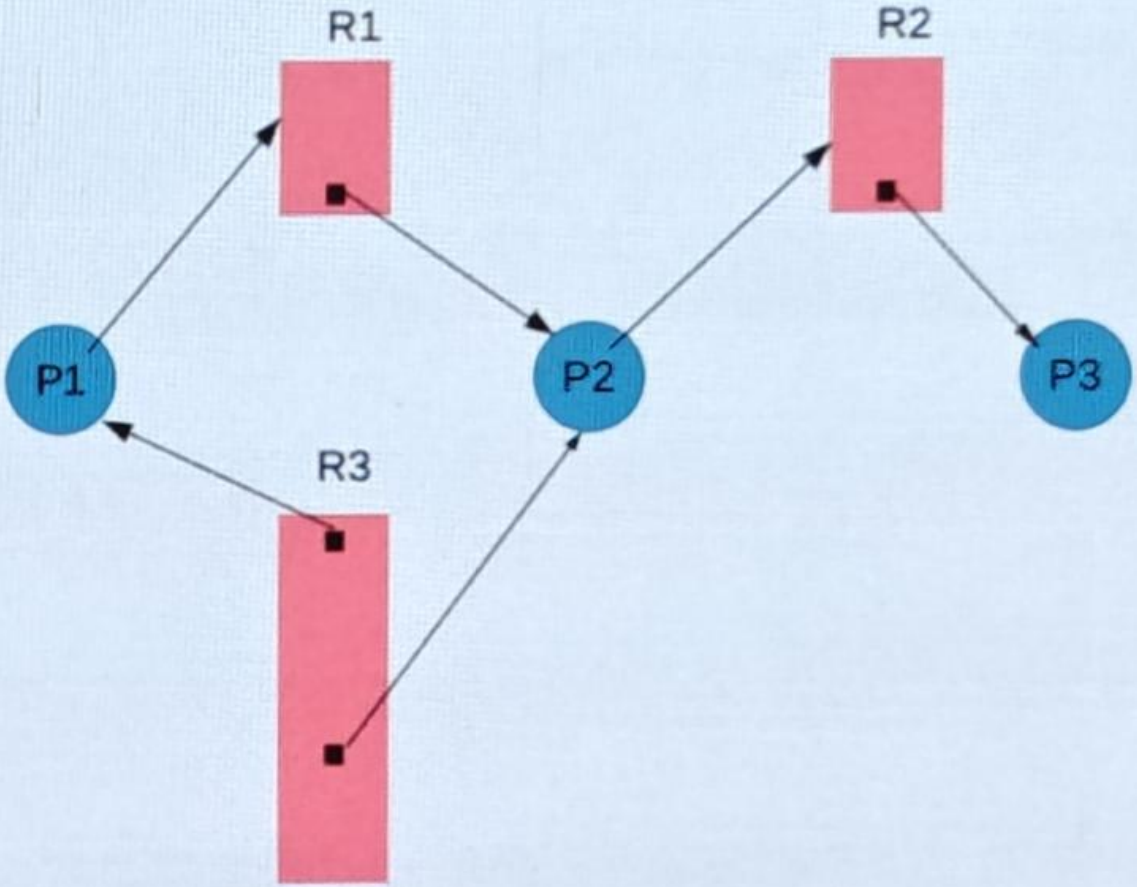


Illustration with RAG /2

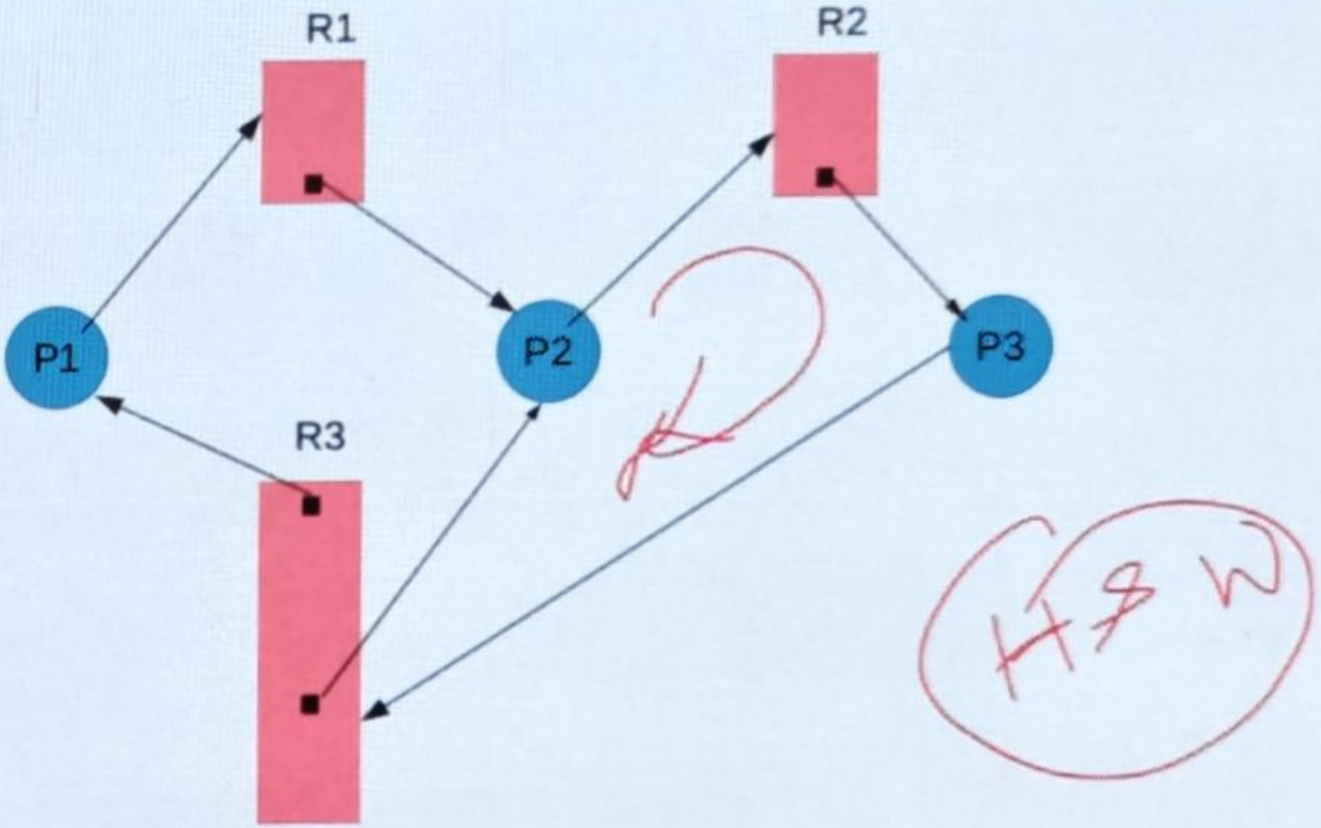
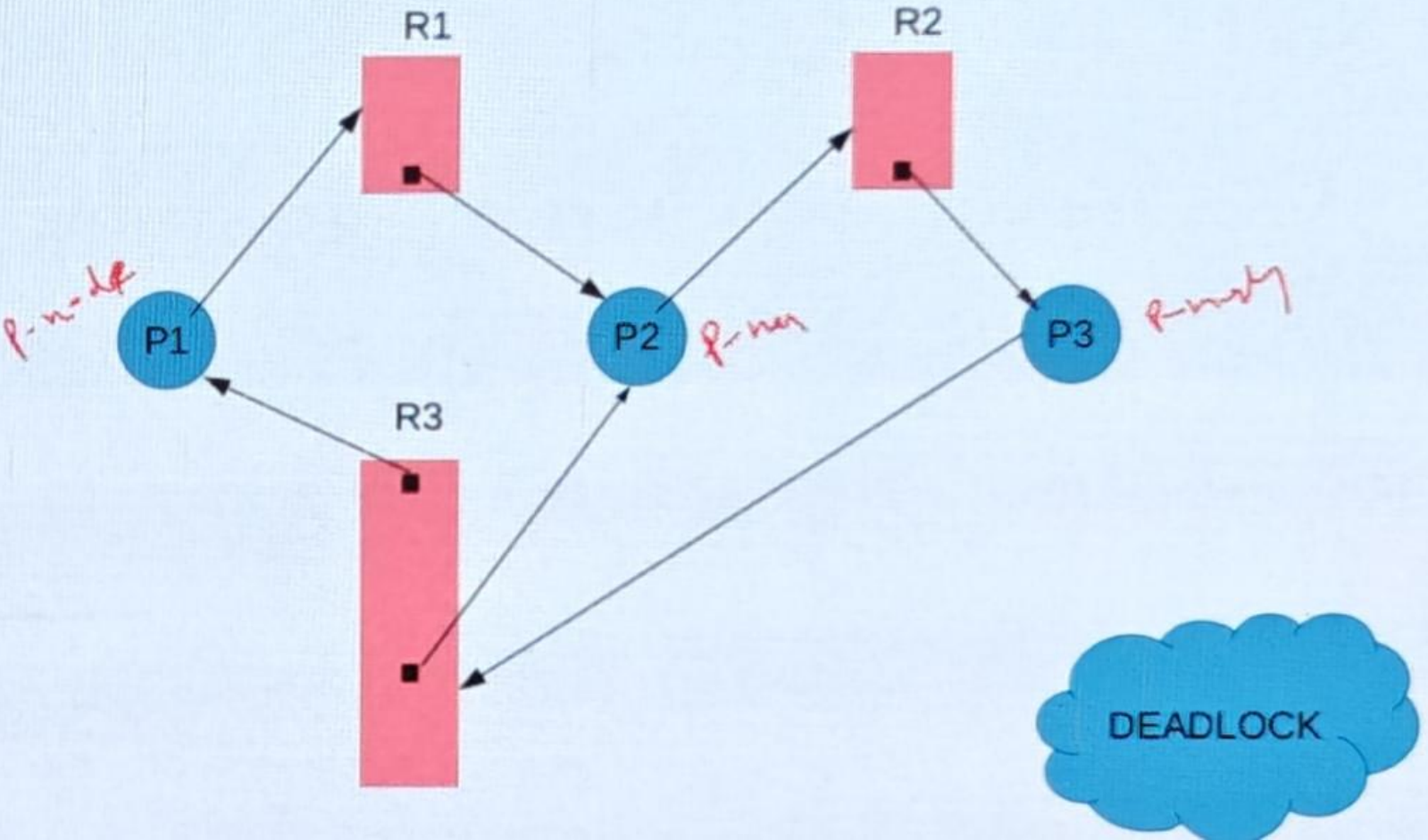


Illustration with RAG /2

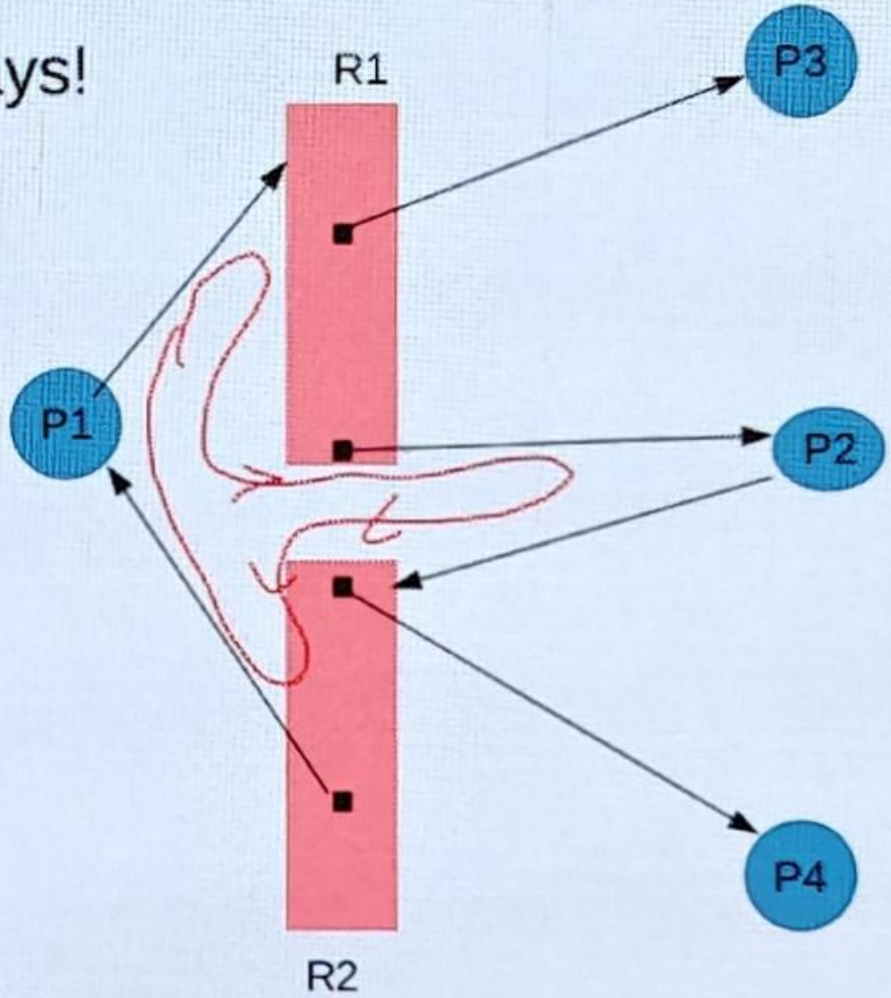


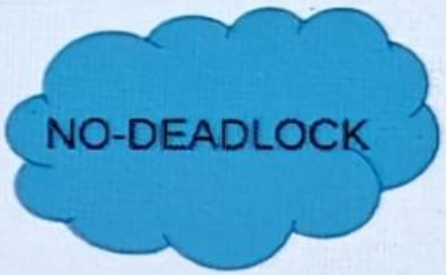
Deduction from RAG

1. If only one resource instance is there for all resources in a set
2. AND If there exists a cycle in the RAG
→ Deadlock!
3. Not always!

Deduction from RAG

Not always!





Deadlock: Solutions

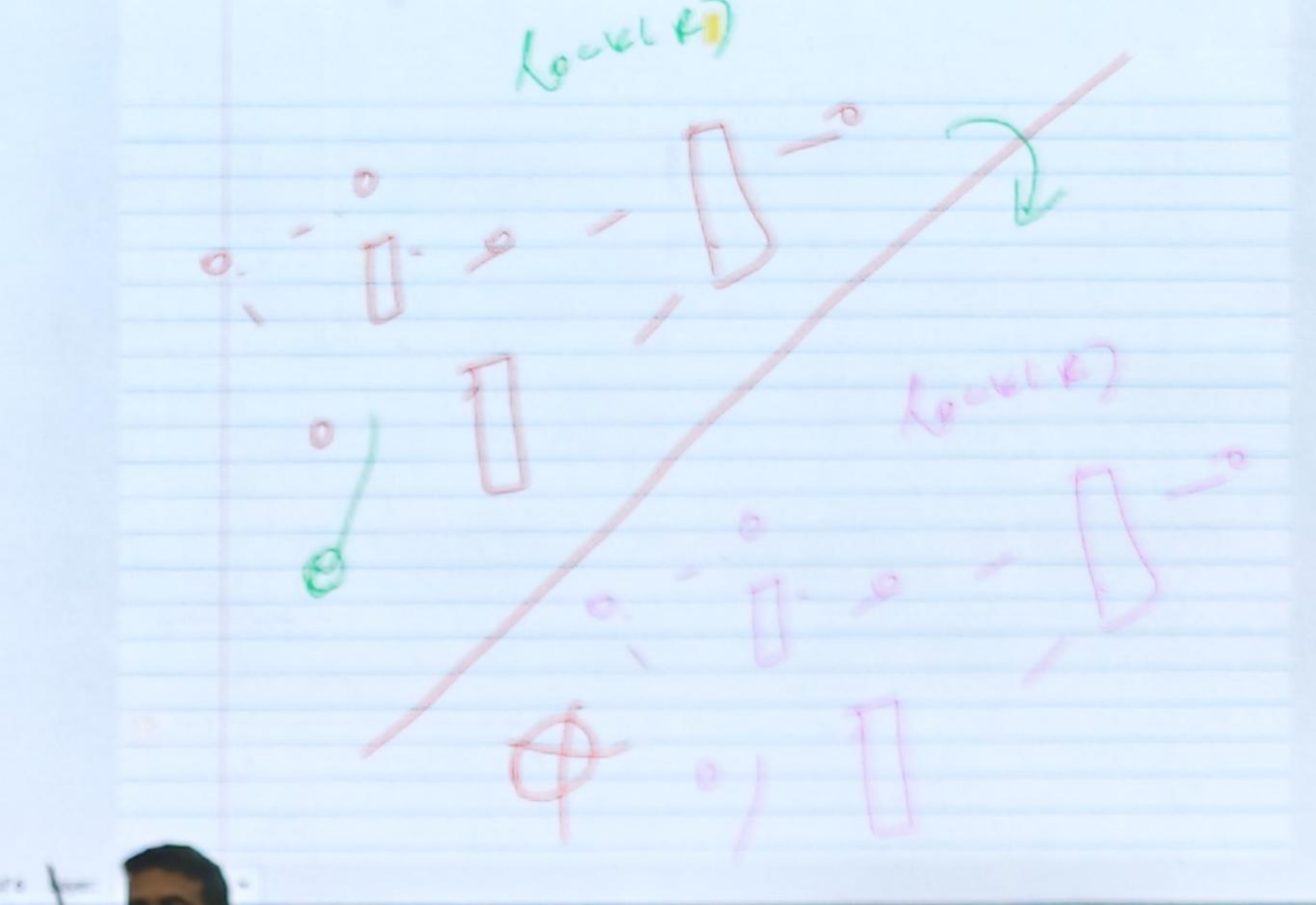
- Follow protocols for resource allocation
- These protocols can provide:
  - Deadlock prevention: one of four criteria shall not hold at every instance
  - Deadlock avoidance: “leading to state” must be avoided, ! DO NOT share any resources!!!!!
  - Deadlock mitigation: identify and break
    - Might lead to LIVELOCK!!!!!
  - Example: Bankers protocol, **Stack Based Priority Ceiling protocol (SBPC)**, etc.

Solution to Deadlock: Stack-based Priotiy Ceiling (SBPC) Protocol

CS303
Autumn 2025
17 Oct 2025

Criteria for Deadlock Occurrence

- Four (w.r.t accessing of shared resources)
  - Mutual Exclusion
  - Hold and wait
  - No preemption (of shared resources before intended unlocking)
  - Circular waiting
- Typically, for deadlock to occur ALL criteria shall hold
- Solutions to DL:
  - Prevention: Any of the criteria if NOT allowed to becomme TRUE, DL can be **prevented**
  - Avoidance: Deadlock can be avoided by identifying when system is in the LEADS-to-DL state i.e. in a state, which would in the next step lands in DL state
  - Identification and Mitigation



Stack-based Priority Ceiling Protocol

- Scheduling policy: Priority-based scheduling
- Protocol approach: process scheduling and resource allocation in an integrated fashion
- Significant points to note:
  - Scheduler follows priroity based scheduling
  - Resources have fixed priorities and these priorities are defined as following
    - If a shared resource R_i would be shared by processes P_j, P_k and P_l
      - For processes P_j, P_k and P_l the pritorities are natural numbers j, k and l, respectively.
      - Then resource R_i priority is given by max([j,k,l])

Stack-based Priority Ceiling Protocol

Defining Rules of the Protocol:

Updating of the Current Ceiling: When all resources are free the CC(t) will be equal to Ω. This CC (t) is updated each time a resource is locked or unlocked.

Scheduling Rule: After a task is released, it starts getting executed only when its base priority value is greater than the current ceiling, as per the priority based sched policy. This means the task will be in ready state until this criteria is met.

Allocation Rule: Whenever a task requests a resource, it is allocated the resource.

It is clear from the scheduling rule that a task is executed only when all the resources it needs during its execution are free, because this happens only when the Base Priority of task is greater than CC (t).Following this no dead lock is possible.

$P_5 : R_1, R_3, R_5$

$P_9 : R_1, R_3$

$P_{17} : R_1, R_4$

$R_1 : P_5, P_9, P_{17}$

P_{20} $\quad pri(R_1) = 17$

RT =

P_9 : {

lock (R_1),

unlock (R_1),

}